Project Proposal (Week 2), Ventilation Fault Detection, Diagnosis and Graceful Degradation

D7065E, Embedded Intelligence at the Edge

Wassim Akachi, Project group 5 (solo)

Summary

Use case	Demand-controlled ventilation that detects, diagnoses and survives its own equipment faults
Pitch	A stuck damper and a room full of people look identical to a CO_2 controller. A model of what the last command should have achieved tells them apart, names the fault, and keeps the air breathable until someone repairs it.
Repository	https://github.com/eduni-pro/202608-d7065e, in subfolder `project/`

1. Use case, and why

The system runs demand-controlled ventilation on a BuildSim office floor: CO_2 rises with occupancy, the controller opens a damper, the air is diluted, the next reading falls. The project proper is the **supervisory loop** on top, which decides whether that control loop is still *working*.

It was chosen because the fault is invisible to the controller, which only observes CO_2: a high reading looks the same whether the damper is stuck shut or the room is simply full. Telling those apart needs a *model* of what the last command should have achieved, so the autonomous part is an inference problem rather than a threshold, and fault injection and degraded operation become the use case itself rather than an experiment bolted on at the end.

2. Scenario

A room fills up and CO_2 climbs past 1000 ppm. The controller opens the damper wide, but it is stuck, so the airflow never changes and CO_2 keeps rising. The monitor saw the same command, expected the level to fall, and instead watches the gap grow until it is too large and too persistent to be noise. Unchanged fan power argues against a blocked filter, a correct overnight decay argues against a drifting sensor, and the monitor publishes a confirmed *stuck damper*. The supervisor degrades that room, over-ventilates its neighbour to hold the occupants below the limit, and raises a work order shown on the 3D view. The diagnosis stays latched even though the CO_2 curve now looks healthy again.

3. What will be built *(rough sketch)*

- **Physics simulator:** advances the CO_2 and occupancy models, applies the injected fault, and writes the resulting sensor values into BuildSim.
- **Sensor and actuator processes:** publish CO_2 and fan power read from BuildSim; validate and apply damper commands back to BuildSim, which stays the single source of building state.
- **Controller:** the inner loop, choosing a damper position per zone from CO_2.
- **Monitor and supervisor:** the outer loop, split in two on purpose. The monitor predicts the response each command should produce, watches the residual and names the fault; it raises events and **never commands anything**. The supervisor acts on a confirmed fault, degrading the zone, compensating from a neighbour and issuing the work order.
- **Storage and dashboard:** readings, commands, residuals and decisions land in a time-series store that feeds both the evaluation and a dashboard overlaying residual and fault state per zone on the 3D view.

Each piece runs as its **own process**, communicating through defined interfaces: a message stream for readings, commands and fault events, BuildSim's API for building state. Splitting monitor from supervisor is deliberate, a crashing monitor must not be able to stop ventilation, and a wrong diagnosis must not be able to command anything by itself.

4. Simulating the building

BuildSim provides the rooms but no physics, so both the air and the people are simulated.

- **Physical processes:** a well-mixed CO_2 balance per zone, production rising with the number of people, dilution set by the airflow the commanded damper position allows, and volume taken from the room's floor area. Commands feed back into the model, so the loop closes. Fan power is derived from the flow, so a blocked filter costs more power for less air.
- **The equipment fault:** injected *between* the command and its physical effect, a stuck damper ignores the command, a blocked filter scales the flow down, and a drifting sensor adds a slow offset at the sensor rather than in the room. This model is invented rather than measured, and the report will say so.
- **Movement of people (occupancy):** a weekday schedule per room type, arrivals, a lunch dip, departures, booked meetings, empty nights and weekends, with randomness so no two days are alike. The empty nights double as the reference condition for checking the sensor: with nobody in the building, CO_2 *must* decay to the outdoor level.

5. The autonomous part

There are two decisions, and only the second is interesting. The **controller** sets each zone's damper position from CO_2 with hysteresis and a freshness check, deliberately a plain rule.

The **monitor** is the autonomous service proper. Starting approach: predict the CO_2 response the command should have produced, take the residual against the measurement, and accumulate it over time (a one-sided CUSUM) so a small persistent bias is caught long before a raw threshold would fire. A short rule tree over residual, fan power and overnight decay separates the three faults, with an explicit *unclassified* branch that alerts a human rather than acting. A learned alternative will be measured against that baseline on identical seeds; if it does not win, the report will say so and keep the baseline.

6. Biggest unknowns & risks

- The prediction wants to know occupancy, the one thing a real building does not. The intended answer is to judge commanded *step responses*, whose shape depends mainly on airflow and volume rather than head-count, but that is unproven under realistic noise.
- Whether the three fault signatures stay separable once sensor noise is realistic, or two of them collapse into one class.
- **Masking:** once the supervisor compensates, CO_2 recovers, the residual shrinks, and a naive monitor clears a fault that is still there. Latching and occasional active probing are the intended answer; this is the risk most likely to bite.
- The fault model and the detector are both mine, so the evaluation measures the detector against my own generator.

7. First step & rough plan

The riskiest assumption is that a residual separates a stuck damper from a busy room at all, so that is the first prototype: one zone, offline, a step response with and without the fault, and a plot of the two residuals. If they do not separate, the use case changes now rather than in week 8. After that, one zone end-to-end to prove the loop closes; monitor and supervisor split into their own processes; more zones, storage and the dashboard; the fault-injection harness; the experiments, detection delay, false alarms and a scaling run to the breaking point; then report and demo.